Target-directed closure and smaller typed checking for iPRA

Exact presentation theorems, explicit rules, and an honest comparison

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

Two distinct refinements are established. First, the recent progressive seven-rule presentation can eliminate a closure using only its target term. At an equation matrix, a literal successor target requests its predecessor instance; other targets request the zero instance. A zero-counter iterator wrapper recovers arbitrary-target closure and equation induction without an addition identity. We prove exact consequence equivalence with the existing orbit presentation, so its whole-language bi-interpretation with iPRA transfers. The matched checker shrinks from 4,553 to 4,532 normalized bytes, but the existing witness-sensitive checker remains smaller at 4,456 bytes.

Second, an independent implementation theorem replaces recursive integer/list typing by a lexical gate before JSON decoding. The complete ordinary Choice-DI checker, including closed-goal validation and exact typing of the entire certificate, falls from 1,039 to 895 normalized bytes (442 to 399 significant tokens; 502 to 453 AST nodes). A more explicit error-reporting version is 963 bytes. The arithmetic and logical rules of these compact checkers are unchanged. Neither result beats the separate 521-byte canonical-goal core or establishes an absolutely simplest axiomatization. New metatheorems are written proofs, supported by executable certificates, not newly Lean-verified results.

1 Result, comparison target, and the boundary of the claim

The requested comparison is against the actual current project, including incoming packages, at the immutable snapshot

`76697262071ada91f29c14d9cf3801a64dacb994.`

The repository is `VladimirReshetnikov/ProveIt`, path `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The README and the four incoming archives were read; their Git blob identities are recorded in `evidence/provenance.json`.[1, 2]

The main result is deliberately divided into two objects.

1. **A mathematical presentation refinement:** the new calculus $\mathcal{T}$ in Section 3. Its primitive closure elimination has one numerical parameter and contains no arithmetic orbit term. It is bi-interpretable with full first-order iPRA, by explicit proof translations and the inherited bridge. Its complete direct checker is `code/target_checker.py`.
2. **A smaller checked implementation of the existing theory:** `code/lexical_compact.py`, the 895-byte typed Choice-DI checker. Its accepted mathematical proof language is preserved by a parser-boundary theorem. It is *not* an implementation of $\mathcal{T}$.

These distinctions prevent assigning the compact implementation's size to the new mathematical calculus. The target rule is simpler than the recent orbit rule in a matched implementation. It does not improve on every existing calculus. The typed checker improves a concrete complete interface, including validation, without deleting a mathematical axiom.

What was already present. The project already contains equational-only transport, direct closure introduction, source-selected elimination, a single witness-sensitive introduction, crossed transport, one-context equality transport, one-hole replacement, and root-sensitive progressive closures. These are not claimed as new here. The incoming `uniform_progress` and `kernel_refinement` packages already attain the coordinate envelope 256 tokens / 278 AST nodes / 521 bytes, using three different programs.[1, 3, 4, 5]

Measurement contract. Every reported module is standalone. Its proof checker, input handling, validation, and final acceptance output are all counted. A proof producer, test driver, normalization script, mathematical report, or theorem database is never imported by a checker. Significant tokens, AST nodes, and UTF-8 bytes are measured after normalization with `ast.unparse(ast.parse(source))` and one trailing newline, removing genuine docstrings from the direct typed controls. Deterministic gzip at level 9 is supplied for information. Measurements use CPython 3.12.14 and zlib 1.3.2; regeneration records the actual versions used.

No lower-bound claim. The smallest exhibited checker is an upper bound on description size. There is no proved lower bound here and no proof that further reductions are impossible. The requested improvement over the entire current mathematical frontier has not been established. The archive preserves the two narrower positive results rather than relabelling them as that stronger result.

2 The unchanged conditional theory

This section specifies the mathematical system whose compact checker is reduced in Section 4. The language has one numerical sort,

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h), \qquad A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

Only u in the iterator body is bound. The ordinary readings, uniform over all formula roots, are

$$Q_y B(y) := B(0) \to \forall y\, B(y), \qquad D_{x,y} B := \exists x (B(x, 0) \to \forall y\, B(x, y)).$$

The contradiction is $0 = S0$. Contexts are finite reusable lists; all substitutions avoid capture. There are exactly the following primitive patterns:

$$\begin{aligned}
&\text{Hyp}: \quad A \in \Gamma \Longrightarrow \Gamma \vdash A;\\
&\text{I0}: \quad \Gamma \vdash I(0, a; u.h) = a;\\
&\text{IS}: \quad \Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u];\\
&\text{Tr}_{\neq 0}: \quad \Gamma \vdash s = t, \quad \Gamma \vdash F(s) \Longrightarrow \Gamma \vdash F(t), \qquad s \not\equiv 0;\\
&\text{QE}: \quad \Gamma \vdash Q_y B, \quad \Gamma \vdash B(0) \Longrightarrow \Gamma \vdash B(t);\\
&\text{DE}: \quad \Gamma \vdash D_{x,y} B, \quad \Gamma, Q_y B(b, y) \vdash A \Longrightarrow \Gamma \vdash A;\\
&\text{DI}_{\text{case}}: \quad \Gamma, B(t, 0) \vdash B(t, Sz) \Longrightarrow \Gamma \vdash D_{x,y} B;\\
&\text{DI}_{\text{prog}}: \quad \Gamma, E(t, z) \vdash E(t, Sz) \Longrightarrow \Gamma \vdash D_{x,y} E, \qquad E \text{ a literal equation.}
\end{aligned}$$

Here F is a native formula template. The source restriction means syntactic inequality with the literal zero term, not numerical nonzeroness. The DE eigenparameter b is fresh for the context, conclusion, and fixed matrix parameters. The DI eigenparameter z is fresh for the context, witness, and fixed parameters. Local assumptions are discharged. There are no other primitive equality, successor, logical, or induction axioms.

The source restriction is the established admissible restriction of the project. With unrestricted transport, these are the inherited Choice-DI rules; source-dispatch recovers all omitted zero-source instances. The existing native/scalar/iPRA bridge therefore applies. [1, 6, 7, 9]

A significant implementation distinction. The 895-byte program preserves *both* DI choices at every witness. In its four-field introduction packet, the false-valued field `u=0` selects atomic progress, whereas `u=[]` selects cases, even at an equation. These are distinct exact JSON types and remain distinct after lexical validation. The expression `len(a) % 2 == u` must not be replaced by the root-only test `len(a) == 2`. Such a replacement would change the theory to a different auxiliary calculus and would require a separate public-language theorem. This is neither the seven-rule witness presentation nor the deterministic guarded presentation.

Bi-interpretation being preserved. The native-to-ordinary interpretation fixes every numerical term and literal equation and uses the two displayed binder readings. Conversely, the ordinary logical operators are finite macros in the native language; Appendix A states them. The scalar ordinary theory has the two iterator equations, numerical coverage, separation, and contextual equation induction. The inherited predecessor, equality reflection, pairing, and primitive-recursion constructions interpret it in iPRA and back with both numerical composite graphs $x = y$. Both formula composites are derivable equivalences for the full language. The present change to input checking changes none of these maps, axioms, or proofs.

3 The target-directed progressive calculus

The arithmetic terms and the progressive interpretation of atomic closures are inherited. The new change concerns elimination: a closure instruction supplies only its desired target term. A syntactic successor target asks for its predecessor instance; every other atomic target asks for the zero instance. An iterator with zero counter is a certified way to request the second mode even when the desired result is a successor. This removes the separate numerical source parameter and the addition term used by the `uniform_progress` proposal.

The construction below has seven primitive inference schemas and the full, unrestricted equation/Q/D formula grammar. Its theorem is exact consequence equivalence with that earlier progressive calculus, including open sequents and every formula of their common language. It is a further presentation refinement, not a new arithmetic foundation or a claim of absolute minimality.

3.1 Complete primitive specification

There is one numerical sort. Terms and formulas are finite syntax trees:

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h), \qquad A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

Only u in h is bound by the iterator. The closure Q binds y; D binds the distinct variables x, y. All substitution is capture-avoiding. Terms are not silently normalized before a rule is

selected. In particular, $I(0, Sr; u.0)$ is an iterator-headed term, although its value is Sr. The contradiction is $0 = S0$.

Call a matrix *atomic* when its outermost constructor is equality, and *compound* when its root is Q or D. Set

$$\eta_B(z) = \begin{cases} z, & B \text{ atomic}, \\ 0, & B \text{ compound}, \end{cases} \qquad \delta_B(t) = \begin{cases} r, & B \text{ atomic and } t \text{ is literally } Sr, \\ 0, & \text{otherwise.} \end{cases} \tag{Target dispatch}$$

These are metasyntactic definitions, not new numerical functions or tests for whether an arbitrary number is zero or a successor.

Contexts are finite reusable lists of formulas. The calculus $\mathcal{T}$ has precisely the following seven primitive inference schemas:

$$\frac{A \in \Gamma}{\Gamma \vdash_{\mathcal{T}} A}\ \text{Hyp}, \qquad \frac{}{\Gamma \vdash_{\mathcal{T}} I(0, a; u.h) = a}\ \text{I0}, \qquad \frac{}{\Gamma \vdash_{\mathcal{T}} I(Sn, a; u.h) = h[I(n, a; u.h)/u]}\ \text{IS},$$

$$\frac{\Gamma \vdash_{\mathcal{T}} s = t \qquad \Gamma \vdash_{\mathcal{T}} p(s) = q(s)}{\Gamma \vdash_{\mathcal{T}} p(t) = q(t)}\ \text{Tr}_{=}, \tag{1}$$

$$\frac{\Gamma \vdash_{\mathcal{T}} Q_y B(y) \qquad \Gamma \vdash_{\mathcal{T}} B(\delta_B(t))}{\Gamma \vdash_{\mathcal{T}} B(t)}\ \text{QE}_{\text{target}}, \tag{2}$$

$$\frac{\Gamma \vdash_{\mathcal{T}} D_{x,y} B(x, y) \qquad \Gamma, Q_y B(b, y) \vdash_{\mathcal{T}} A}{\Gamma \vdash_{\mathcal{T}} A}\ \text{DE}, \tag{3}$$

$$\frac{\Gamma, B(w, \eta_B(z)) \vdash_{\mathcal{T}} B(w, Sz)}{\Gamma \vdash_{\mathcal{T}} D_{x,y} B(x, y)}\ \text{DI}_{\text{root}}. \tag{4}$$

Here $p(v), q(v)$ in (1) are numerical term templates. In DE, b is fresh for Γ, A and the fixed parameters of B. In DI, z is fresh for Γ, w and the fixed parameters of B. The displayed local assumptions are discharged. Weakening, contraction, exchange, cut, and numerical substitution are admissible operations, not additional primitive instructions.

There are no other primitive logical axioms, ordinary logical connectives, quantifier rules, successor axioms, equality axioms, or induction schemas. The literal test in (Target dispatch) is part of the complete specification; replacing it by a test of the value of t would define a different system.

3.2 The two closure interfaces and induction

Weakening, exchange, contraction, fresh renaming, and assumption cut follow by the usual induction on finite derivations. For cut, replace the appropriate hypothesis leaves and rename eigenparameters first. Numerical substitution needs a separate check and is proved below, after the elementary closure construction it uses.

Put

$$W(t) := I(0, t; u.0), \qquad 1 := S0. \tag{5}$$

The state variable is fresh and unused. The equation $W(t) = t$ is an I0 instance, while $W(t)$ is never syntactically a successor.

Lemma 3.1 (Equality and derived closure introduction)**.** *Reflexivity, symmetry, transitivity, and numerical congruence are derivable in $\mathcal{T}$. In addition,*

$$\frac{\Gamma, B(\eta_B(z)) \vdash_{\mathcal{T}} B(Sz)}{\Gamma \vdash_{\mathcal{T}} Q_y B(y)}\ \text{QI}_{\text{root}}$$

is admissible, with z fresh for the retained context and fixed parameters.

Proof. Use $W(a) = a$ as both premises of equational transport with equation template $v = a$ to obtain $a = a$. From $s = t$ and $s = s$, transport with template $v = s$ gives $t = s$. The usual equation templates then give transitivity and numerical congruence; no transport through a compound formula is used.

For QI, apply DI at witness 0 to $D_{r,y}B(y)$, with r absent from B. Eliminate this result by DE. Its opened assumption is $Q_yB(y)$, independent of the fresh witness parameter, so Hyp supplies the desired conclusion of the DE minor. □

Lemma 3.2 (One step and arbitrary-target closure)**.** *For every equation $E(y)$ and every term t, the following rules are admissible under arbitrary retained contexts:*

$$Q_yE(y),\ E(t)\ \vdash_{\mathcal{T}}\ E(St), \qquad Q_yE(y),\ E(0)\ \vdash_{\mathcal{T}}\ E(t). \tag{Atomic interfaces}$$

For every compound matrix B, ordinary closure elimination is primitive:

$$Q_yB(y),\ B(0)\ \vdash_{\mathcal{T}}\ B(t). \tag{Compound interface}$$

Proof. For one-step progress, choose target St in (2); its selected source is exactly t.

For arbitrary-target atomic closure, first choose target $W(t)$. Since it is iterator-headed, its selected source is 0. QE gives $E(W(t))$ from Q_yE and $E(0)$. The I0 equation $W(t) = t$ and primitive equational transport then give $E(t)$. This derivation uses no numerical substitution metatheorem and no compound transport.

For compound B, the definition of δ_B is identically 0, so the displayed rule is a direct QE instance. □

Corollary 3.3 (Contextual equation induction)**.** *For every literal equation $E(y)$,*

$$\frac{\Gamma \vdash_{\mathcal{T}} E(0) \qquad \Gamma, E(z) \vdash_{\mathcal{T}} E(Sz)}{\Gamma \vdash_{\mathcal{T}} E(t)}$$

is admissible, where z is fresh for the retained context and fixed parameters.

Proof. The step premise gives $Q_yE(y)$ by QI. Apply the arbitrary-target atomic closure from Lemma 3.2 to the supplied base. □

This is the complete bootstrap. It requires neither a predecessor, a numerical case split, an identity for addition, nor an assumption that a syntactically nonsuccessor term denotes zero. The wrapper in (5) is a constant-size term constructor in the proof translation; its numerical value need not be computed by the checker.

Lemma 3.4 (Capture-free numerical substitution)**.** *If $\Gamma \vdash_{\mathcal{T}} A$, then $\Gamma\sigma \vdash_{\mathcal{T}} A\sigma$ for every capture-free numerical substitution σ.*

Proof. Induct on the derivation, freshly renaming discharged eigenparameters. All rules other than QE commute with substitution in their displayed forms: in particular, the outer formula constructor, and hence η_B, is preserved.

Consider QE with matrix B and target t. If B is compound, both before and after substitution the source is 0. If B is an equation and $t = Sr$, then $t\sigma = S(r\sigma)$ and the selected source remains $r\sigma$. If B is an equation and neither t nor $t\sigma$ is a syntactic successor, both selected sources are 0. These cases translate to primitive QE instances.

The remaining case is an atomic matrix with a nonsuccessor target t whose substitution becomes a successor. The translated premises are $Q_y(B\sigma)(y)$ and $(B\sigma)(0)$, whereas a primitive QE at

$t\sigma$ would request its predecessor instance. Apply the already proved arbitrary-target atomic closure instead. It yields $(B\sigma)(t\sigma)$ from exactly those translated premises. Lemma 3.2 was established without the present substitution theorem, so this repair does not introduce a circular dependency. □

3.3 Transport through the whole formula language

Lemma 3.5 (Full transport and homogeneous binder congruence)**.** *For every native formula template $F(v)$,*

$$s = t,\ F(s) \vdash_{\mathcal{T}} F(t).$$

If matrices B, C are uniformly interderivable and have the same root class (both equations or both compound), their Q closures are interderivable, as are their D closures.

Proof. For transport, induct on the formula template. An equation uses primitive equational transport. For $Q_y E(v, y)$ with an equation matrix, introduce the target closure by atomic QI: move its temporary $E(t, z)$ back to $E(s, z)$, use one-step elimination for the retained source closure, and move $E(s, Sz)$ forward to $E(t, Sz)$. Both moves are equation transports.

For $Q_y B(v, y)$ with a compound matrix, introduce the target by compound QI. The induction hypothesis moves its local base $B(t, 0)$ to $B(s, 0)$; compound QE at Sz gives $B(s, Sz)$; the induction hypothesis moves this back to $B(t, Sz)$. These transports concern the strictly smaller matrix.

For $D_{x,y} B(v, x, y)$, eliminate the source at a fresh witness b and introduce the target at that same witness. Use the induction hypothesis to move the local introduction premise from parameter t to s. Apply one-step elimination if the matrix is atomic and ordinary compound closure elimination otherwise. Move the successor conclusion back by the induction hypothesis, and close DI and DE. Every transport used inside this construction concerns a strictly smaller formula.

For homogeneous congruence, the same constructions use the two supplied matrix conversions in place of numerical transport. The root-class condition ensures that the two DI or QI premises use the same source $\eta(z)$. In the D case, eliminate the source binder and reuse its fresh witness for the target binder. □

The qualification "homogeneous" matters. An equation can be equivalent to a compound formula without its progressive closure being equivalent to the compound formula's conditional-universal closure. No unrestricted congruence across these two root classes is asserted.

3.4 Exact comparison with the inherited orbit calculus

Let $\mathcal{R}$ be precisely the seven-rule calculus in the incoming `uniform_progress` report.[3] It has the same terms, formulas, Hyp, I0, IS, equational transport, DE, and root-directed DI as $\mathcal{T}$. Only its closure elimination differs. Write

$$A(a, n) := I(a, n; u.Su).$$

Its elimination rule is

$$\frac{Q_y B(y) \qquad B(\eta_B(a))}{B(A(\eta_B(a), n))}. \tag{Orbit QE}$$

The formulas and terms in this comparison have their literal common syntax; the translations between $\mathcal{T}$ and $\mathcal{R}$ are identity maps on them, with changes only to their proof certificates.

Dependencies on the $\mathcal{R}$ side. The inherited report first proves contextual equation induction inside $\mathcal{R}$: at source 0, (Orbit QE) gives $E(I(0,t;u.Su))$, and I0 plus equational transport gives $E(t)$. It then proves the elementary addition identities by equation induction, extracts $QE, E(a) \vdash E(Sa)$ using $A(a,1) = Sa$, and derives full formula transport by structural induction. In particular, its compound transport is available before the comparison below. None of those inherited proofs uses the new target-directed rule.

Lemma 3.6 (Elementary addition identities in $\mathcal{T}$)**.** *The calculus $\mathcal{T}$ proves*

$$A(0,n) = n, \qquad A(Sa,n) = SA(a,n),$$
$$A(a,0) = a, \qquad A(a,Sn) = SA(a,n), \qquad A(a,n) = A(n,a).$$

Proof. The first two are computation instances. Induct on a for $A(a,0) = a$, then on a for $A(a,Sn) = SA(a,n)$. For commutativity, induct on a: the base is $n = A(n,0)$ and the successor step is $A(Sa,n) = SA(a,n) = SA(n,a) = A(n,Sa)$. All inductions are on equations, using Corollary 3.3. □

Theorem 3.7 (Identical consequence relations)**.** *For every context Γ and every formula F in the full common equation/Q/D grammar,*

$$\Gamma \vdash_{\mathcal{T}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{R}} F.$$

Both directions are effective transformations of finite derivations.

Proof. For $\mathcal{T} \to \mathcal{R}$, only a target-directed QE node needs to be replaced. If its matrix is an equation and its target is Sr, use the inherited one-step rule at r. If its matrix is an equation and its target is not a successor, its minor is $E(0)$; use orbit QE at source 0 and initializer t, then I0 and equation transport. For a compound matrix, its minor is again the zero instance; the same orbit specialization gives $B(I(0,t;u.Su))$. Use the already established full transport of $\mathcal{R}$ and I0 to obtain $B(t)$. The last step is why compound transport must be placed before this comparison.

For $\mathcal{R} \to \mathcal{T}$, only orbit QE needs replacement. For a compound matrix, target-directed QE from $B(0)$ may target the exact term $A(0,n)$, so it gives the required conclusion immediately.

For an equation matrix, retain $Q_y E(y)$ and $E(a)$. Equation induction on k proves $E(A(k,a))$: the base follows from $A(0,a) = a$, and the successor step follows from one-step elimination at $A(k,a)$ and $A(Sk,a) = SA(k,a)$. Specialize at $k = n$. Commutativity from Lemma 3.6 and equation transport convert $E(A(n,a))$ to $E(A(a,n))$, exactly the old target. All other inference nodes translate literally. The local constructions retain arbitrary side assumptions and preserve fresh-eigenparameter conditions after renaming. □

The addition identities in this last comparison explain how to simulate an old orbit certificate. They are not required by the new primitive rule, the new checker, the induction bootstrap, or the direct proof of its ordinary soundness.

3.5 Ordinary interpretation and both full-language composites

The ordinary reading is exactly that of $\mathcal{R}$:

$$\begin{aligned}
[\![s = t]\!] &= (s = t), \\
[\![Q_y E]\!] &= \forall y\,(E(y) \to E(Sy)) \quad \text{if } E \text{ is an equation}, \\
[\![Q_y B]\!] &= [\![B(0)]\!] \to \forall y\,[\![B(y)]\!] \quad \text{if } B \text{ is compound}, \\
[\![D_{x,y} B]\!] &= \exists x\,[\![Q_y B]\!].
\end{aligned}$$

For completeness, the new QE is directly sound in the inherited scalar ordinary theory T_{eq}. At an atomic successor target it uses one progress implication. At an atomic nonsuccessor target it uses equation induction from the zero instance. At a compound matrix it is ordinary conditional-universal elimination. Its soundness does not rely on the target term denoting any particular kind of number. The other rules have the same soundness proofs as the inherited progressive calculus.

Here is the precise composition with the existing native/iPRA bridge. Let $\mathcal{N}$ be the native calculus whose closures are uniformly read as $B(0) \to \forall y B(y)$. Define the inherited padding

$$G_v(s = t) := (I(v, 0; u.s) = I(v, 0; u.t)), \qquad L(E) := D_{r,v} G_v E,$$

using fresh unused u, r, v where required, and put $P(E) = L(E)$ for an equation and $P(B) = B$ for a compound formula. The inherited forward translation $F : \mathcal{N} \to \mathcal{R}$ fixes equations and terms and satisfies

$$F(Q_y B) = Q_y P(FB), \qquad F(D_{x,y} B) = D_{x,y} P(FB).$$

The reverse translation $J : \mathcal{R} \to \mathcal{N}$ fixes equations and terms. At atomic closures it uses the native logical macros for progressivity:

$$J(Q_y E) = \forall_{\mathcal{N}} y \, (E(y) \to_{\mathcal{N}} E(Sy)),$$
$$J(D_{x,y} E) = \exists_{\mathcal{N}} x \, \forall_{\mathcal{N}} y \, (E(x, y) \to_{\mathcal{N}} E(x, Sy)).$$

At compound matrices it recurses without changing the binder: $J(Q_y B) = Q_y JB$ and $J(D_{x,y} B) = D_{x,y} JB$. The native macros and the full proofs of this inherited translation theorem are retained in the source material; they are not new axioms of $\mathcal{T}$.

The inherited result[3] proves derivation preservation in both directions and both contextual formula composites,

$$A \dashv\vdash_{\mathcal{N}} JF(A), \qquad B \dashv\vdash_{\mathcal{R}} FJ(B),$$

for every old A and every progressive-language B. In particular, its second composite is a theorem for all unguarded atomic closures as well; it is not restricted to a canonical public fragment.

Theorem 3.8 (Whole-domain bi-interpretability with iPRA)**.** *The ordinary realization of $\mathcal{T}$ is equiconsistent and bi-interpretable with full first-order intuitionistic primitive recursive arithmetic. Both numerical domains are whole, numerical equality is actual equality, and both numerical composite isomorphisms are the identity graph $x = y$. Every formula of the unrestricted native grammar has the stated round trip.*

Proof. By Theorem 3.7, the identity formula translation between $\mathcal{T}$ and $\mathcal{R}$ preserves their entire consequence relations. Read the same syntactic forward map F as taking values in $\mathcal{T}$, and read the same reverse map J as starting from $\mathcal{T}$. Translate the inherited $\mathcal{R}$ derivations by that theorem. This proves preservation and the composite $B \dashv\vdash_{\mathcal{T}} FJ(B)$, while the $\mathcal{N}$ composite is unchanged.

Compose these maps with the existing native/scalar/iPRA bi-interpretation.[9] The scalar terms, domain, and equality are unchanged by the present maps; therefore the inherited numerical identity composites are unchanged. The contradiction $0 = S0$ is fixed, giving equiconsistency by the effective proof translations. This is an external theorem about the two formal theories, not an assertion within iPRA of its own consistency. □

Scope of the result. The new mathematics is the target-directed primitive elimination, its constant-size wrapper recovery of arbitrary-target closure, and the exact full-language presentation theorem above. The progressive interpretation, padding translations, ordinary logical macros, and the arithmetic bridge to iPRA are inherited and explicitly identified. The proofs in this section are written metatheory, not a newly compiled Lean verification. In the matched standalone implementation, the new checker improves the orbit checker by the recorded 21 normalized bytes, 11 significant tokens, and 8 AST nodes. The already existing witness-sensitive checker remains smaller in that comparison. These facts support a local refinement of the orbit presentation; they do not establish a new minimum over every current proposal.

4 A smaller complete integer/list input boundary

The second result applies to the unchanged conditional calculus of Section 2, not to the progressive reading of Section 3. Its reference is the incoming `one_context` package's complete `strict/code/checker_typed.py`.[5] That module checks every certificate value recursively:

```
def j(a):
    return type(a) is int or (type(a) is list and all(j(b) for b in a))
```

It also repeats exact integer/list checks in the canonical public-goal validator. These tests can be charged once at the text boundary.

Let Σ consist of the characters

$$[\ ]\ ,\ -\ 0\ 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9$$

and JSON's four whitespace characters: space, tab, carriage return, and newline. The explicit new gate is

```
raw = open(0).read()
if raw.strip('[],-0123456789 \t\r\n'):
    raise ValueError('Only integer/list certificate syntax is allowed')
data = json.loads(raw)
```

Lemma 4.1 (Lexical characterization of recursive exact typing)**.** *Suppose the unchanged Python JSON parser successfully decodes a finite input text r to a. Then every character of r lies in Σ if and only if a satisfies the recursive predicate j above.*

Proof. A JSON string necessarily contains quotation marks, an object contains braces, and the spellings of true, false, and null contain letters. None of these characters is in Σ. A nonintegral JSON number contains a decimal point or exponent marker, also excluded. Python's additional NaN and Infinity spellings contain excluded letters. Consequently a successfully parsed value over Σ is an integer or a list whose children have the same property. JSON integer decoding produces exact Python integers, not booleans. Structural recursion proves $j(a)$.

Conversely, a successfully parsed integer is spelled with digits and possibly a leading minus; a list uses brackets and commas and the spellings of its elements. If $j(a)$ holds, no other JSON value kind can occur. Whitespace admitted by the JSON grammar is precisely the four listed characters. Thus the whole spelling lies in Σ. □

Lemma 4.2 (The use of `strip` is exact)**.** *A finite string r satisfies `r.strip(chars) == ''` precisely when every character of r belongs to `chars`.*

Proof. If all characters are permitted, the complete string is stripped. If some character is forbidden, stripping either end cannot remove that character, so the result is nonempty. This test does not claim that `strip` deletes forbidden interior characters. □

The JSON parser remains responsible for punctuation and number syntax. For example, a leading-zero integer, an unterminated list, adjacent values, or a bare minus can pass the alphabet gate but still fails parsing. Negative zero is harmless: it decodes to integer zero in both programs. Strings containing digit escapes are rejected as strings; there is no evaluation of certificate text as Python code.

4.1 Removing duplicate tests without changing the goal grammar

After this gate and successful JSON decoding, every value is known to be an exact integer or exact list. For such values,

$$a * 0 = [] \quad \Longleftrightarrow \quad a \text{ is a list.}$$

The exact-type tests in the goal validator can therefore be replaced by this already-used arithmetic/list distinction. The canonical grammar, allowed arities, repeated-triple condition for Q, numerical-coordinate restrictions, and the kernel's initial scope opening are all unchanged.

For reference, the compact encoding uses arrays of arity two for an equation, arity one for D, and a triple of identical entries for Q. Numerical zero is integer zero, successor is unary, and an iterator is ternary. The inherited opening operation accounts for binder scope using arity-dependent dummy slots. The public validator continues to permit only zero and odd coordinates at least three at numerical leaves; the opener enforces their scope. This report does not enlarge the public language with administrative zero aliases.

Theorem 4.3 (Exact preservation of the ideal certificate language)**.** *Replace recursive j by the lexical gate, and replace the redundant exact-type tests in the public validator as just described. Keep every proof-kernel expression unchanged. The old and new programs accept the same finite certificates in their ideal, unbounded-resource semantics.*

Proof. On a valid JSON input rejected by j, Lemma 4.1 makes the new lexical gate reject as well. On a valid JSON input passing j, the gate passes and the decoded tree is the identical tree. The new public validator agrees with the old validator on that tree because its replaced predicates agree on every integer/list node. Thereafter the programs run the same kernel on the same goal, retained resources, fresh-name floor, and instruction stream.

Invalid JSON cannot lead to acceptance in either program. A root value that is not a certificate list fails the subsequent unchanged extraction operations. Unused prefix instructions remain inert for proof checking, but their types remain constrained, since the lexical gate examines the entire input. This proves both directions. □

The theorem concerns accepted mathematical certificates, not equality of every exception message or machine-resource failure. The old recursive typing traversal can exhaust the Python stack on a deeply nested unused prefix that the lexical gate does not traverse. Parser limits, memory limits, and proof-recursion limits remain operational limitations; resource exhaustion never counts as acceptance.

4.2 Explicit and compact drivers

The explicit version retains a descriptive typing error and is 963 normalized bytes. The recommended size profile additionally uses the following Boolean gate:

```
q = open(0).read()
q = not q.strip('[],-0123456789 \t\r\n') and json.loads(q).pop
a = q()
```

For forbidden input, short-circuit evaluation assigns the Boolean `False` to `q`; the immediate call raises an exception, so the program cannot reach its acceptance output. For permitted input, `q` is precisely the same list-pop operation as before. The explicit variant is useful when a descriptive failure is preferred. Neither program uses assertions, so optimized Python does not remove a logical check.

A separate frozen control applies the same Boolean-driver factoring to the old recursive predicate. It is 963 bytes, compared with 895 bytes for the Boolean lexical version. Thus the lexical contribution can be measured independently of shortening the error branch. The complete 1039-to-895 change is not all assigned to a single optimization.

5 Measurements and executable evidence

5.1 The same full typed certificate contract

Complete module	Tokens	AST	Bytes	Gzip 9
Inherited recursive typed control	442	502	1,039	547
Explicit lexical gate	404	455	963	549
Boolean lexical gate (recommended)	399	453	895	512

The recommended module reduces bytes by 144 (13.86%), tokens by 43 (9.73%), and AST nodes by 49 (9.76%). Gzip decreases by 35 bytes in that profile; in the more explicit profile it increases by two bytes. Compression is not the optimization objective. The exact additionally factored control and its measurements are included in the machine-readable results.

5.2 The same direct typed rule-checker framework

Calculus	Tokens	AST	Bytes	Gzip 9
Adopted native eight-rule control	1,235	1,407	4,692	1,199
Existing witness-sensitive seven-rule	1,173	1,334	4,456	1,198
Existing progressive orbit $\mathcal{R}$	1,206	1,366	4,553	1,211
Target-directed $\mathcal{T}$	1,195	1,358	4,532	1,220

The target-directed change saves 21 bytes, 11 tokens, and 8 AST nodes against its exact orbit comparator. It increases gzip by nine bytes. It remains 76 bytes, 22 tokens, and 24 AST nodes larger than the existing witness-sensitive calculus. Replacing the separate source/seed orbit instruction removes a mathematical dependency and a certificate parameter; that gain is local, not a new overall minimum.

5.3 The compact canonical-goal frontier is a different contract

Existing profile	Tokens	AST	Bytes	Gzip 9
Token profile	256	283	594	346
AST profile	260	278	602	344
Byte profile	263	300	521	327

These are three complete programs with a prescribed canonical public goal contract; they do not perform the full public validation and whole-certificate exact typing charged to the 895-byte module. No single program in this table has the combined tuple $(256, 278, 521)$. The new work does not improve any of these three compact minima.

5.4 What was checked

The reproduction script records exact hashes, Python and zlib versions, normalization conventions, positive and negative certificates, parser attacks, and process outcomes in `evidence/`. It checks that the compact proof kernel is syntactically identical before and after the lexical change, and exercises both false-valued DI modes. Whole-input typing controls include booleans, null, strings, floats, exponent syntax, objects, NaN/Infinity, malformed JSON, and forbidden data in an otherwise unused prefix. Runs use normal and optimized Python.

The direct target-calculus controls exercise successor and nonsuccessor targets, a compound closure at a successor target, the wrapper recovery of an arbitrary target, induction with a used induction hypothesis, fresh-variable restrictions, malformed terms, and negative examples. The delivered reproduction records 652 subprocess executions with zero mismatches, 3,068 lexical-gate checks per profile, 30 closed target-calculus cases, five open-sequent controls, and 600 independent named-syntax substitution/lift checks. The frozen evidence files record the expected outcomes. These tests support the implementations. They do not replace the universal equivalence proofs or establish an absolute lower bound.

6 Assessment and reproducibility

The smaller fully typed checker is a justified implementation improvement of an existing complete iPRA presentation. The new target-directed calculus is a justified mathematical refinement of the recent orbit presentation, with a simpler induction bootstrap and exact full-language round trips. The existing witness-sensitive presentation remains smaller in the matched direct comparison. This is the precise extent to which the requested simplification was obtained.

Other investigated routes did not supply a stronger result. Making the atomic guard implicit simplifies the public datatype but introduces an elaboration cost that loses against a similarly factored guarded validator. A stopped-clock construction in the self-seeded arithmetic precursor gives a useful diagonal equation, but does not prove successor injectivity. Neither failed route is used as an axiom deletion here. The audit notes record the limits; no independence or impossibility theorem is inferred from the failed searches.

Archive contents. `report.pdf` and its standalone `report.tex` contain the present results. `foundations.pdf` and `foundations.tex` retain the attributed inherited native/ordinary/arithmetic and progressive-closure proofs. `code/` contains complete new checkers; `reference/` contains exact controls and upstream attribution; `tests/` contains certificates and producers; `evidence/` contains measurements, verification receipts, and audits. Compressed normalized checker sources are supplied alongside the measurement artifacts.

Run the standard-library-only verification script from the extracted directory, and compile either standalone LaTeX source with a normal TeX installation:

```
python3 reproduce.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error foundations.tex
pdflatex -interaction=nonstopmode -halt-on-error foundations.tex
```

Acceptance means normal process exit and exactly `ACCEPT` plus one newline. Every exception is rejection. The new metatheorems have been independently reviewed in writing. Lean and Lake were unavailable, so no new compiled Lean result or machine verification of the Python kernel is claimed.

A Ordinary logical macros and the arithmetic bridge

The following finite macros belong to the uniform conditional native language of Section 2. They must be expanded there before applying the translation F into the progressive language. They are inherited from `NativeGate.lean` and `NativeLogic.lean`.[8] Put $C(v,a,b) = I(v,a;u.b)$, with u unused. Define the equation/formula selector recursively by

$$\begin{aligned} U_v(s,t;r=w) &= (C(v,s,r) = C(v,t,w)),\\ U_v(s,t;Q_yB) &= Q_yU_v(C(y,0,s),C(y,0,t);B),\\ U_v(s,t;D_{x,y}B) &= D_{x,y}U_v(C(y,0,s),C(y,0,t);B). \end{aligned}$$

Set $G_vA = U_v(0,0;A)$. Define the general selector by

$$\begin{aligned} M_v(s=t,B) &= U_v(s,t;B),\\ M_v(Q_yA,B) &= Q_yM_v(A,G_yB),\\ M_v(D_{x,y}A,B) &= D_{x,y}M_v(A,G_yB). \end{aligned}$$

Variables are freshened throughout. The native rules derive G_0A, $G_{Sz}A \dashv\vdash A$, $M_0(A,B) \dashv\vdash A$, and $M_{Sz}(A,B) \dashv\vdash B$. With $p(y) = I(y,0;u.C(u,1,0))$, the logical operations are

$$\begin{aligned} \bot_* &= (0=1), & A \Rightarrow_* B &= Q_yM_y(A,B),\\ A \wedge_* B &= Q_yG_yM_{p(y)}(A,B), & \exists_* x\, A(x) &= D_{x,y}G_yA(x),\\ A \vee_* B &= \exists_* v\, M_v(A,B), & \forall_* x\, A(x) &= A(0) \wedge_* Q_xA(x). \end{aligned}$$

These are macros, not extra primitive schemas. Implication elimination uses the selector at 1; existential elimination uses DE, the zero gate, and QE at 1; the two parity values recover conjunction; numerical cases supply disjunction elimination. From $0=1$, selector computations give arbitrary term equality, and structural recursion gives ex falso for every formula. The complete inherited arguments are compiled in the companion foundations document.

The ordinary scalar theory T_{eq} uses intuitionistic first-order equality logic, the two iterator equations, coverage $\forall n(n=0 \vee \exists m\, n = Sm)$, separation $0 = S0 \to \bot$, and contextual induction on literal equations. The inherited chain constructs predecessor first, then equality reflection and quantifier-free induction, then pairing and projections, and finally a compiler for every fixed primitive recursive definition. The opposite translation interprets scalar iteration by primitive recursion. The two numerical composites are provably the identity; the formula composites are provable equivalences.[9]

No variable ranges over primitive-recursive program descriptions, and no total universal evaluator is postulated. The compiler is an external recursion on each fixed finite description. This preserves the intended strength of iPRA rather than silently adjoining a stronger evaluator.

References

[1] V. Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt README, pinned commit `76697262071ada91f29c14d9cf3801a64dacb994`. https://github.com/VladimirReshetnikov/ProveIt/blob/76697262071ada91f29c14d9cf3801a64dacb994/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.

[2] ProveIt, *ConciseIPRA/docs/incoming*, four archives at the same pinned commit. Blob identities are reproduced in `evidence/provenance.json`. https://github.com/VladimirReshetnikov/ProveIt/tree/76697262071ada91f29c14d9cf3801a64dacb994/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/docs/incoming.

[3] *A uniform introduction rule for intuitionistic primitive recursive arithmetic*, incoming `ipra_uniform_progress_2026-11-11.zip`, blob `2b8392d6036daf0272b8697b549557aefc9e1cad`. Full mathematical core retained in the companion foundations document.

[4] *A smaller checked presentation of iPRA*, incoming `ipra_kernel_refinement_2026-11-11.zip`, blob `d00ab53c90daec469ff17e1b36e9eede5be92779`.

[5] *One term context for equality*, incoming `ipra_one_context_2026-11-11.zip`, blob `46e1344d8ce4bd91c696f0eefe951a306ea6d3eb`. The exact complete typed module is frozen as the lexical comparison control.

[6] ProveIt, *source_dispatch*, source-restricted transport and its admissibility proof, under `ConciseIPRA/Research/source_dispatch/` at the pinned commit.

[7] ProveIt, *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`, reproduced with attribution in the companion foundations document.

[8] ProveIt, *NativeGate.lean*, *NativeLogic.lean*, *NativeInherited.lean*, and *NativeZeroSource.lean*, under `ConciseIPRA/Lean/ConciseIPRA/` at the pinned commit.

[9] ProveIt, *NativeBiInterpretation.lean*, *BiInterpretation.lean*, and the scalar predecessor, equality reflection, and primitive-recursion sources. The relevant existing endpoints are `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff`.